

Dokumentacja Użytkownika

Uruchamianie programu

Razem z kodem źródłowym dostarczamy zbudowany plik wykonywalny, można też zbudować program samemu używając:

```
cmake -B build
cmake --build build
```

Aby rozpocząć, wystarczy wywołać plik wykonywalny w terminalu:

```
./build/Badania0peracyjne
```

Następnie w kolejnych krokach podawane są parametry symulacji oraz solwera. Wszystkie parametry mają domyślne wartości, które można zatwierdzić wciskając 'Enter'.

Tryb nieinteraktywny (argumenty wiersza poleceń)

Program można również uruchomić podając parametry jako argumenty w terminalu. Pozwala to na szybkie odpalenie eksperymentu bez przechodzenia przez kreatora. Przykład wywołania ze wszystkimi możliwymi argumentami:

```
./build/Badania0peracyjne --width 10 --height 10 --preset random --penalty
checkerboard --starting_states 10 --duplications 100 --max_iterations 1000 --
mutations 40 --patience 10
```

Konfiguracja Problemu

W pierwszej fazie definiowany jest sam problem:

- **Szerokość planszy (Width):** Określa, ile komórek w poziomie liczy plansza, na której układane będą klocki. Domyślnie 10.
- **Wysokość planszy (Height):** Określa, ile komórek w pionie liczy plansza. Domyślnie 10.
- **Zestaw klocków (Preset):** Decyduje o dozwolonych kształtach klocków dostępnych dla algorytmu podczas rozwiązywania. Do wyboru są trzy zestawy (wprowadź cyfrę 0, 1 lub 2):
 - **0 (Tetris):** Zestaw klasycznych klocków znanych z gry Tetris (kształty T, L oraz S). Zestaw ten utrudnia ciasne dopasowywanie z uwagi na swoje skomplikowane i zróżnicowane kształty.
 - **1 (Simple):** Zestaw składający się wyłącznie z klasycznych klocków domino o wymiarach 2x1. Przydatny do testowania podstawowej poprawności algorytmu.
 - **2 (Random):** Unikalny zestaw pięciu losowo wygenerowanych, spójnych, ale bardzo nieregularnych klocków (o rozmiarach od 2 do 5 komórek).
- **Mapa kar (Penalty):** Definiuje stały rozkład nagród i kar przydzielanych za zajęcie danego pola planszy przez blok klocka. Silnie wpływa na punkty i wymusza na algorytmie omijanie lub preferowanie pewnych obszarów (wprowadź cyfrę od 0 do 5):
 - **0 (Uniform):** Wszystkie kary równe 1. Algorytm skupi się wyłącznie na jak najgęstszym pokryciu całej planszy, niezależnie od lokalizacji klocków.
 - **1 (Bad Diagonal):** Wysokie kary punktowe (-10) narzucone na obie główne przekątne planszy. Wymusza na algorytmie układanie klocków z dala od środkowego krzyża X.
 - **2 (Good Diagonal):** Premie punktowe (+10) na przekątnych. Algorytm z dużym prawdopodobieństwem spróbuje jako pierwsze zapełnić właśnie oba ukosy planszy.
 - **3 (Bad Corners):** Kary punktowe (-10) umiejscowione wyłącznie w czterech zewnętrznych rogach planszy.
 - **4 (Good Corners):** Punktowe bonusy (+10) w czterech rogach.
 - **5 (Checkerboard):** Naprzemienny, regularny wzór szachownicy, w którym co drugie pole planszy to kara (-5), a wszystkie sąsiednie pola wokół niego to premia (+5).

Konfiguracja Solvera

W drugiej fazie precyzyjnie dostrajasz mechanikę samego algorytmu ewolucyjnego, co ma kluczowy wpływ na skuteczność oraz szybkość działania:

- **Liczba stanów początkowych (Starting states):** Definiuje rozmiar podstawowej populacji, która podlega mutacjom i selekcji. Wyższa wartość oznacza lepszą eksplorację przestrzeni i większą różnorodność na starcie (co minimalizuje ryzyko utknięcia w optimum lokalnym), ale wiąże się ze znacznie większym zużyciem pamięci RAM i powolniejszą pojedynczą iteracją. Domyślnie 10.
- **Duplikacja stanu (Single state duplications):** Określa stopień rozrodczości – czyli ile kopii (dzieci) jest wytwarzanych i mutowanych w każdej iteracji od jednego, przetrwałego stanu bazowego z poprzedniego pokolenia. Wyższe wartości intensyfikują tzw. przeszukiwanie lokalne wokół obiecujących rozwiązań. Domyślnie 100.
- **Maksymalna liczba iteracji (Max iterations):** Stanowi ogólny, twardy limit cykli (pokoleń) dla algorytmu ewolucyjnego. Działa jako mechanizm ochronny kończący obliczenia po zadanym czasie. Domyślnie 1000.
- **Mutacje na iterację (Mutations per iteration):** Parametr definiujący głębokość mutacji. Oznacza ile pojedynczych, wylosowanych akcji (takich jak dodanie klocka, usunięcie klocka, przesunięcie lub obrót) jest nakładanych jedna po drugiej na każde pojedyncze dziecko w obrębie jednej iteracji. Niska liczba to bardzo ostrożna eksploracja bliska stanowi początkowemu iteracji, z kolei wysoka powoduje drastyczne zmiany i chaotyczne wędrowanie rozwiązań. Domyślnie 40.
- **Cierpliwość (Patience):** Kluczowy czynnik warunkujący przedwczesne zakończenie optymalizacji (tzw. early stopping). Definiuje on z góry, przez ile dokładnie następujących po sobie iteracji globalny, najwyższy wynik nie może ulec żadnej poprawie, żeby algorytm uznał swoje rozwiązanie za ostateczne i przerwał symulację (zamiast dotrzeć do limitu maksymalnej liczby iteracji). Domyślnie 10.

Generowanie wizualizacji

Podczas działania algorytmu stany eksportowane są do plików '.csv'.

Aby ułatwić i zautomatyzować analizę, dołączono dedykowane narzędzie w języku Python. Zamieni ono zrzucone stany na kolorowe obrazki ilustrujące zachowanie solvera.

`python visualize.py`

Skrypt znajdzie wygenerowane w ostatniej iteracji pliki .csv i przekształci je na łatwe w odbiorze pliki graficzne w formacie .png, eksportując je do folderu `plots/`. Na renderingu niezajęte przez klocki komórki siatki oznaczone są kolorem czarnym, natomiast różne rodzaje klocków różnymi kolorami. Na każdym polu zaznaczone jest również kara.